

MongoDB Tutorial

What is Database?

A database is an organized collection of structured information or data, typically stored electronically in a computer system. It allows for efficient storage, retrieval, management, and updating of data, usually managed by a [Database Management System](#) (DBMS). Databases are crucial for modern applications, supporting secure, fast access and data integrity.

Types of Databases (SQL vs NoSQL)?

Databases are generally categorized into two main paradigms: SQL (Relational) and NoSQL (Non-relational). Each is designed with different architectural priorities for data structure, scalability, and consistency.

SQL Databases (Relational)?

SQL databases store data in a structured, tabular format using rows and columns. They are ideal for applications where data relationships are complex and data integrity is paramount.

- **Structure:** Uses a predefined schema; data must be organized into fixed tables and columns before entry.
- **Scalability:** Typically, **vertically scalable**, meaning they handle increased load by adding more power (CPU, RAM, SSD) to a single server.
- **Transactions:** Follow strict ACID properties (Atomicity, Consistency, Isolation, Durability) to ensure reliable transaction processing.
- **Querying:** Uses Structured Query Language (SQL), a standardized language for complex joins and data manipulation.
- **Examples:** MYSQL, SQL, PostgreSQL, Oracle Database.

NoSQL Databases (Non-Relational)?

NoSQL ("Not Only SQL") databases store data in flexible formats such as documents, graphs, or key-value pairs. They are optimized for high-volume, unstructured data and rapid development.

- **Structure:** Uses a dynamic schema; data can be stored without a predefined format, allowing for varied attributes within the same collection.
- **Scalability:** Designed for horizontal scalability, allowing them to handle massive traffic by adding more servers (nodes) to a cluster.

- **Consistency:** Often follows the BASE model (Basically Available, Soft state, Eventual consistency), prioritizing availability and speed over immediate consistency.
- **Querying:** Varies by database type; may use JSON-like syntax, APIs, or specialized languages like Cypher for graphs.
- **Examples:** MongoDB, Redis, Cassandra, Neo4j.

What is NoSQL?

MongoDB is an open-source, document-oriented NoSQL database designed to store and manage large volumes of data efficiently using a flexible, JSON-like document model.

- Stores data in BSON format for efficient handling of JSON-like documents.
- Supports dynamic schemas, making it suitable for unstructured or semi-structured data.
- Scalable and easy to learn, widely used in modern web and mobile applications.
- Open-source and freely available on Linux, Windows, and macOS.

Install MongoDB & MongoDB Compass:

- Download MongoDB Community Server:
 - <https://www.mongodb.com/try/download/community>
- MongoDB Compass:
 - When Install the server there compass already here.
- MongoDB Shell Download:
 - <https://www.mongodb.com/try/download/shell>

Note: After Installing set environment variables: server and mongosh

JSON VS BSON?

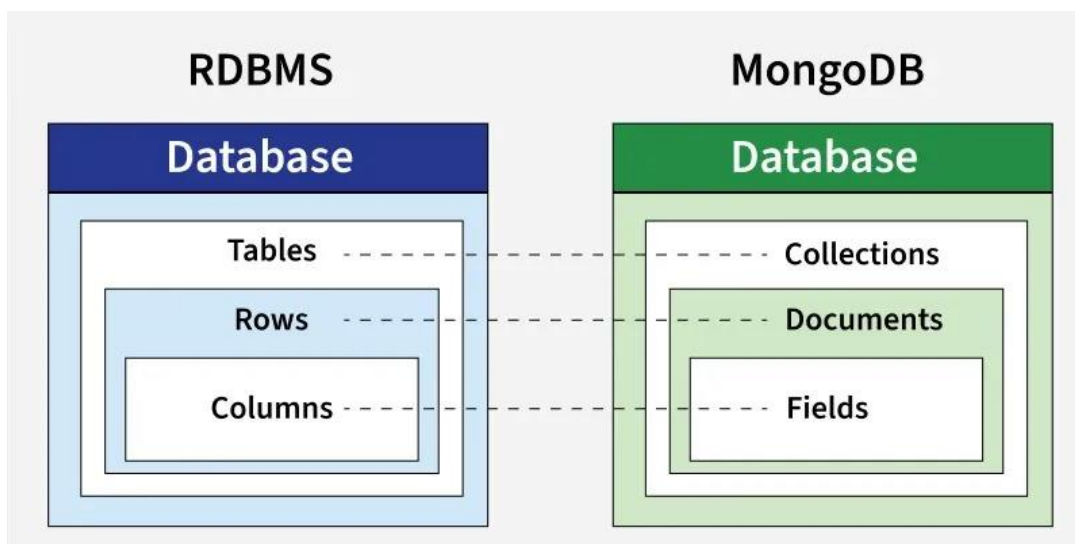
JSON and BSON are data interchange formats built on key-value pairs, but JSON is a human-readable, text-based format for general data exchange, while BSON is a binary-encoded format optimized for efficient data storage and faster machine processing, primarily used by MongoDB.

Feature	JSON	BSON
Format Type	Text-based (human-readable)	Binary-based (machine-readable)
Readability	Yes, easily readable by humans	No, not human-readable
Data Types Supported	Strings, numbers, booleans, arrays, null	All JSON types + Date, Binary, ObjectId, and others
Space Efficiency	Less efficient for complex data types	More efficient due to binary encoding
Performance	Slower parsing and retrieval	Faster parsing and retrieval, especially for large data
Use Case	Data exchange between client and server	Data storage, especially in MongoDB
Compression	No built-in compression	More compact due to binary encoding
Support for Large Data	JSON may struggle with very large data	BSON supports large documents and binary data natively

Advantages of BSON:

- **Efficient Storage:** BSON's binary format reduces data size compared to JSON, especially when handling large datasets.
- **Speed:** The binary encoding allows for faster reading and writing of data compared to the textual format of JSON.
- **Extended Data Types:** BSON supports more advanced data types like Date, Binary, and ObjectId, which are crucial for handling special cases such as timestamps and unique identifiers.

RDBMS VS MongoDB?



Create a Database In MongoDB:

In MongoDB, a database is created only when data is inserted.

use database

Check Existing Databases:

Command: show dbs

This will return the name of the currently selected database

Command: db

Create a Collection in MongoDB:

A collection in MongoDB is a group of schema-less documents, similar to a table in a relational databases, created explicitly or automatically on insertion.

Example: db.createCollection('Students')

InsertOne Method:

Inserting a document automatically creates the collection if it doesn't exist.

db.Students.insertOne({Name: "Farhan", age: 23})

insertMany() Method:

insertMany() method inserts multiple documents in one operation, improving performance over single inserts

Example:

db.Students.insertMany([{name:"Providers", country:"PK"}, {name:"Ali", age:20}])

View Existing Collections

After inserting data, list all collections using the show collections command.

Command: show collections

Drop Existing Collections & Database

Command: db.students.drop()

db.dropDatabase()

InsertOne Method:

Inserting a document automatically creates the collection if it doesn't exist.

```
db.Students.insertOne({Name: "Farhan", age: 23})
```

insertMany() Method:

insertMany() method inserts multiple documents in one operation, improving performance over single inserts

Example:

```
db.Students.insertMany([{name:"Providers", country:"PK"}, {name:"Ali", age:20}])
```

Query Embedded:

Read Record:

This command is used for getting all data.

Example: db.users.find()

Example: db.users.find({city: {\$in : ["Karachi", "Lahore"]}})

Example: db.users.find({\$and: [{name: "Ali"}, {age: {\$lt: 20} }] })

Example: db.users.find({age: 18})

Example: db.users.find().sort({ name: 1})

Example: db.users.find().limit(5)

Read Specific Record:

This command gets only one record.

Example: db.users.findOne({ age: 25 })

Example: db.users.findOne({ name: "Ali"})

Update Record:

The `updateOne()` method in MongoDB updates the first document that matches a specified condition in a collection.

updateOne():

Example: `db.users.updateOne( { name: "Ali" }, { $set: { age: 26 } } )`

updateMany()

Example: `db.users.updateMany( { city: "Karachi" }, { $set: { country: "Pakistan" } } )`

Example: `db.users.updateMany({ age: 19 }, $set: {isvote: true})`

Delete Record:

deleteOne()

Deleting first document from the student collection using an empty filter {}:

Example: `db.users.deleteOne({})`

Example: `db.users.deleteOne({ name: "Ali" })`

Example: `db.users.deleteOne({ age: { $lt: 19 } })`

deleteMany()

The `deleteMany()` method in MongoDB is used to remove all documents from a collection that match a specified filter.

Example: `db.users.deleteMany({})`

Example: `db.users.deleteMany({city: "Karachi"})`

Example: `db.users.deleteMany({ age: { $lt: 25 } })`

Comparison Query Operators:

MongoDB comparison query operators filter documents by matching exact values or selecting value ranges across numeric, string, and date fields, helping retrieve relevant results efficiently.

- **Exact match:** Use `$eq`, `$ne` to match or exclude values.
- **Multiple values:** Use `$in`, `$nin` to match or exclude from a list of values.
- **Range queries:** Use `$gt`, `$gte`, `$lt`, `$lte` to filter ranges.
- **Type support:** Works with numbers, strings, and dates.
- **Better filtering:** Narrows results for faster, accurate queries.

`$nin` operator:

`$nin` matches none of the values specified in an array.

Example: `db.users.find({name: {$nin: ["ali","Ahmed"]}})`

`$in` operator:

`$in` matches any of the values specified in an array.

Example: `db.users.find({name: {$in: ["ali","Ahmed"]}})`

`$lt` operator:

`$lt` matches values of the fields that are less than a specified value.

Example: `db.users.find({salary: {$lt: 20000}})`

`$eq` operator:

`$eq` matches the values of the fields that are equal to a specified value.

Example: `db.users.find({city: {$eq: "Karachi"}})`

`$ne` operator:

`$ne` matches all values of the field that are not equal to a specified value.

Example: `db.users.find({city: {$ne: "Karachi"}})`

`$gt` operator:

`$gt` matches values of the fields that are greater than a specified value.

Example: `db.users.find({age: {$gt: 18}})`

\$gte operator:

\$gte matches values of the fields that are greater than equal to a specified value.

Example: `db.users.find({age: {$gte: 18}})`

\$lte operator:

\$lte matches values of the fields that are less than equal to a specified value.

Example: `db.users.find({age: {$lte: 18}})`

Logical Operators:

MongoDB logical query operators combine multiple conditions in a single query to enable flexible, precise filtering for advanced searches and data analysis.

\$and operator:

It Joins query clauses with a logical AND. Returns documents that match all the conditions.

Command: `db.users.find({$and: [{gender: "male"}, {city: "Karachi"}]})`

\$or operator:

The \$or operator combines multiple conditions, where at least one of the conditions must be true.

Command: `db.users.find({$or: [{gender: "male"}, {city: "Karachi"}]})`

\$nor operator

It Joins query clauses with a logical NOR. Returns documents that fail to match all the conditions.

Command: `db.users.find({$nor: [{gender: "male"}, {city: "Karachi"}]})`

\$not operator:

It Inverts the effect of a query expression. Returns documents that do not match the query expression.

Command: `db.users.find({salary: {$not: {$gte: 25000}} })`

Element Operators:

The Element Operators in the MongoDB return the documents in the collection which returns true if the keys match the fields and datatypes. There are mainly two Element operators in MongoDB:

1. \$exists Operator

Filters documents based on whether a specified field is present or missing, helping handle optional fields and validate schema-less data in MongoDB.

Command: `db.users.find({ age: { $exists: true } })`

Command: `db.users.find({ age: { $exists: false } })`

2. \$type Operator

Filters documents by matching the BSON data type of a field's value (e.g., string, number, array), useful for validating schema-less data in MongoDB.

Command: `db.users.find({ age: { $type: "int" } })`

Command: `db.users.find({ name: { $type: "string" } })`

Projection Operators:

Retrieve only the name and age fields

Command: `db.users.find({}, { name: 1, age: 1 })`

Command: `db.users.find({}, { age: 0 })`

Command: `db.users.find({}, { name: 1, _id: 0 })`

Schema Validation in MongoDB:

MongoDB schema validation helps keep your data organized and correct. With MongoDB validation rules, we can set up guidelines for what data can be stored in our database. This makes it easier to ensure that all the information we save is reliable and useful.

Example Command:

```

db.createCollection("Students", {
  validator: {
    $jsonSchema: {
      bsonType: "object",
      required: ["name", "id", "age", "department"],
      properties: {
        name: {
          bsonType: "string",
          description: "Name must be a string."
        },
        id: {
          bsonType: "int",
          description: "id must be an integer."
        },
        age: {
          bsonType: "int",
          minimum: 10,
          description: "Age must be an integer greater
than or equal to 10."
        },
        dept: {
          bsonType: "string",
          description: "Department must be a string."
        }
      }
    }
  }
});

```

Indexing in MongoDB:

Indexing in MongoDB is a crucial feature that enhances query processing efficiency. Without indexing, MongoDB must scan every document in a collection to retrieve the matching documents and leading to slower query performance.

Indexes are special data structures that store information about the documents in a way that makes it easier for MongoDB to quickly locate the right data.

Why is Indexing important in MongoDB:

MongoDB provides a method called `createIndex()` that allows users to create an index. The key determines the field on the basis of which we want to create an index and 1 (or -1) determines the order in which these indexes will be arranged(ascending or descending).

Indexing improves the performance of:

- Find queries (db.collection.find()).
- Range queries (e.g., queries with <, >, <=, >= operators).
- Sorting (e.g., db.collection.find().sort()).
- Aggregation operations involving filtering, grouping, and sorting.

Create Index in MongoDB:

To create an index, MongoDB provides the createIndex() method. The method requires us to specify the field(s) to index and the order (ascending or descending). We can also specify optional parameters to customize the index creation.

Command: db.users.createIndex({ name: 1 });

Command: db.users.createIndex({ age: -1 });

Compound Index (Multiple Fields):

Command: db.users.createIndex({ name: 1, age: -1 })

Unique Index:

Command: db.users.createIndex({ email: 1 }, { unique: true })

Index for Range Queries

Command: db.users.find({ age: { \$gt: 20, \$lt: 30 } })

Text Index in MongoDB

Command: db.articles.createIndex({ content: "text" })

Command Query: db.articles.find({ \$text: { \$search: "mongodb" } })

Drop Index in MongoDB:

Command: db.users.dropIndex("name_1")

Command: db.users.dropIndex({ name: 1 })

Drop All Index in MongoDB:

Command: db.users.dropIndexes()

View All Index in MongoDB:

Command: db.users.getIndexes()

Import Data JSON in MongoDB

MongoDB's powerful mongoimport tool allows you to easily import data from external files like JSON into your MongoDB collections.

First Download MongoDB Tool:

first download the MongoDB database tools .msi file from [here](#). Here, we are going to download it for Windows.

Install

Add path in environment variable from MongoDB tool bin

Import Data:

Users.json:

```
[
{ "name": "Ali", "age": 25 },
{ "name": "Sara", "age": 22 },
{ "name": "Ahmed", "age": 30 }
]
```

Command: mongoimport -d mydb -c users d:/users.json --jsonArray

Command: mongoimport -d mydb -c users d:/users.json

Command: mongoimport --db mydb --collection users --file users.json --jsonArray

Command: mongoimport --db mydb --collection users --file users.json --jsonArray --drop

--drop this flag first drop the collection and then import this data

Backup Data in MongoDB

Exporting data from MongoDB is a common requirement for sharing data with external applications or performing data analysis using different tools. MongoDB provides a command-line tool called mongoexport to facilitate this process.

Command: mongodump --out C:/backup

Command: mongodump -d mydb -o C:/backup

Command: mongodump --db mydb --out C:/backup

Command: mongodump -d mydb -c users -o D:/backup

Restore Data in MongoDB

In MongoDB, mongorestore utility is used to restore the backup data. It restores the binary backup created by mongodump utility(i.e., BSON data dumps). It can restore either an entire database backup or a subset of the backup.

Command: mongorestore --dir C:/backup

Command: mongorestore --d office C:\backup\mydb

Command: mongorestore --db office --drop C:\backup\mydb

Command: mongorestore --d office --c users C:\backup\mydb/users.bson

Basic Database using Relation:

Department Collection:

```
{  
  _id: ObjectId(),  
  name: "HR",  
  status: "active"  
}
```

Employee Collection

```
{  
  _id: ObjectId(),  
  name: "Ali",  
  salary: 50000,  
  departmentId: ObjectId("department_id_here")  
}
```

Create:

```
db.departments.insertOne({  
  name: "IT",  
  status: "active"  
})
```

Create Employee

```
db.employees.insertOne({
  name: "Ali",
  salary: 60000,
  departmentId: ObjectId("PUT_DEPARTMENT_ID_HERE")
})
```

All Departments: db.departments.find()

All Employees: db.employees.find()

Employee with Department:

```
db.employees.aggregate([
  {
    $lookup: {
      from: "departments",
      localField: "departmentId",
      foreignField: "_id",
      as: "department"
    }
  }
])
```

Employee With Department using \$unwind

```
db.employees.aggregate([
  {
    $lookup: {
      from: "departments",
      localField: "departmentId",
      foreignField: "_id",
```

```
    as: "department"  
  }  
},  
{ $unwind: "$department" }  
])
```